VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



LE HUY VU – 524H0039
NGUYEN VIET ANH – 524h0003

MIDTERM ESSAY

MOBILE APPS DEVELOPMENT

HO CHI MINH CITY, 2026
VIETNAM GENERAL CONFEDERATION OF LABOR

TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



LE HUY VU – 524H0039
NGUYEN VIET ANH – 524H0003

MIDTERM ESSAY

MOBILE APPS DEVELOPMENT

Advised by
Dr. Le Van Vang

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Mr. Le Van Vang**, our instructor and mentor, for his valuable guidance and dedicated support throughout the mid-term report of our project on **"Optimizing Image Loading, Caching, and Displaying in Android Applications"**.

He has been extremely helpful and patient in providing us with constructive feedback and practical insights to refine our technical approach. Furthermore, he encouraged us to dive deeper into performance analysis tools and optimization techniques – such as asynchronous image loading, memory management, and caching strategies – to enhance our application's efficiency and responsiveness.

We have learned a great deal from his expertise and experience in mobile software engineering. We are truly honored and privileged to have him as our supervisor.

Ho Chi Minh city, 2nd November 2026.

Author

(Signature and full name)

Vu

Le Huy Vu

Anh

Nguyen Viet Anh

DECLARATION OF AUTHORSHIP

We hereby declare that this is our own project and is guided by Mr. Le Van Vang; The content research and results contained herein are central and have not been published in any form before. The data in the tables for analysis, comments and evaluation are collected by the main author from different sources, which are clearly stated in the reference section.

In addition, the project also uses some comments, assessments as well as data of other authors, other organizations with citations and annotated sources.

If something wrong happens, We'll take full responsibility for the content of my project. Ton Duc Thang University is not related to the infringing rights, the copyrights that We give during the implementation process (if any).

Ho Chi Minh city, 28th October 2026

Author

(Signature and full name)

Vu

Le Huy Vu

Anh

Nguyen Viet Anh

TABLE OF CONTENT

TABLE OF CONTENT 1

LIST OF FIGURES 4

LIST OF TABLES 5

CHAPTER 1. INTRODUCTION 6

1.1 Motivation 6

1.2 Objectives 6

1.3 Scope 6

1.4 Research Subjects 7

CHAPTER 2. THEORETICAL BACKGROUND 8

2.1 Image Loading and Display Mechanisms 8

2.2 Asynchronous Image Loading 8

2.3 Coil Library 8

2.4 Image Caching Mechanisms 9

2.5 Image Formats 9

2.6 Bitmap and the Risk of Out Of Memory 9

2.7 Lazy Loading and Image-List Display 10

2.8 Image Optimization Techniques 10

2.9 Performance Measurement Tools 11

CHAPTER 3. APPLICATION ANALYSIS AND DESIGN 12

3.1 Application Description 12

3.2 Non-Functional Requirements 13

3.3 General Architecture 14

3.4 Image Loading Workflow 15

3.5 Comparison Between Baseline and Optimized Versions 16

CHAPTER 4. TESTING AND EVALUATION METHODOLOGY 17

4.1 Testing Objectives 17

4.2 Test Cases 17

4.2.1 TC01 – First Gallery Launch 17

4.2.2 TC02 – Reopen the Gallery 17

4.2.3 TC03 – Fast Scrolling 17

4.2.4 TC04 – High-Resolution Images 17

4.2.5 TC05 – Network Disconnection 17

4.2.6 TC06 – Invalid URLs or Slow Network 19

4.2.7 TC07 – Different Image Dataset Sizes 19

4.2.8 TC08 – Before-and-After Optimization Comparison 19

4.3 Evaluation Metrics 20

4.4 Experimental Conditions 21

4.5 Benchmark Results 22

4.6 Analysis of Results 23

4.6.1 Outstanding Cache Efficiency 24

4.6.2 Significant UI Smoothness Improvement and Threading Optimization 24

4.6.3 Drastic Memory Footprint Reduction via Downsampling 25

4.6.4 Scalability and Robust Error Handling 25

4.6.5 Technical Trade-Offs Analysis 26

CHAPTER 5. CONCLUSION 26

CHAPTER 6. Link 27

CHAPTER 7. REFERENCES 27

LIST OF FIGURES

Figure 3.1 Grid View 12

Figure 3.2 Detail View 12

Figure 3.3 System Architecture Diagram 14

Figure 3.4 Loading Workflow Diagram 15

Figure 4.1 TC05 – [List] Base 18

Figure 4.2 TC05 – [List] Optimized 18

Figure 4.3 TC05 – [Detailed] Base 18

Figure 4.4 TC05 – [Detailed] Optimized 18

LIST OF TABLES

Table 4-1 Base version benchmark results 22

Table 4-2 Optimized function benchmark results.....23

CHAPTER 1. INTRODUCTION

1.1 Motivation

Images are an important component of many mobile applications, especially photo galleries, social networks, and online content-viewing applications. Loading and displaying a large number of images can directly affect responsiveness, memory usage, and user-interface smoothness.

Without proper optimization, an application may experience problems such as slow image loading, unnecessary image re-downloads, interface stuttering while scrolling, excessive RAM consumption, or Out Of Memory (OOM) errors.

Therefore, this topic focuses on studying and applying techniques for optimizing image loading, caching, and display in Android applications. A photo gallery application is developed to evaluate the effectiveness of these techniques through practical tests.

1.2 Objectives

- Understand image-loading and image-display mechanisms in Android applications.
- Study how to use the Coil [1] library for asynchronous image loading.
- Explore in-memory and disk-based caching mechanisms.
- Apply image resizing, on-demand loading, and placeholder techniques.
- Build a photo gallery application that can display a large number of images retrieved from URLs.
- Evaluate optimization effectiveness by comparing the application before and after optimization.

1.3 Scope

This topic focuses on an Android application developed using Kotlin. The application displays images in a grid and allows users to tap an image to view it at a larger size.

Images are retrieved from URLs. The application does not require an album-management system, image editing, or complex image-processing features. The main focus is on loading performance, caching, memory management, and image display.

Tests are conducted using datasets containing 100, 500, and 1,000 images to evaluate the application's ability to handle different workloads.

1.4 Research Subjects

- Image-loading mechanisms in Android.
- The Coil image-loading library.
- Kotlin Coroutines [2] and asynchronous processing.
- RAM cache and disk cache.
- Bitmaps, image dimensions, and memory usage.
- Lazy Loading and image-list optimization.
- Performance measurement tools in Android Studio

CHAPTER 2. THEORETICAL BACKGROUND

2.1 Image Loading and Display Mechanisms

When an application loads an image from a URL, the system needs to perform several steps, such as sending a network request, receiving image data, decoding the data into a Bitmap, and displaying it in the user interface

This process can consume processing time, network bandwidth, and memory. If too many large images are loaded simultaneously, the application may respond slowly or consume excessive resources

This issue is especially noticeable in a photo gallery application because many images are displayed at the same time and users may scroll through the list quickly

2.2 Asynchronous Image Loading

Asynchronous image loading allows data retrieval and image processing to take place without blocking the main user-interface thread (Main Thread)

If network operations or intensive image decoding are performed on the Main Thread, the interface may freeze or stutter. Therefore, an application should use a dedicated library such as Coil to manage image-loading and display operations

Kotlin Coroutines also support asynchronous task processing in Android, helping developers organize code in a more readable and manageable way

2.3 Coil Library

Coil is an image-loading and display library for Android. It supports Kotlin and integrates mechanisms such as asynchronous loading, caching, and image-request management

In the photo gallery application, Coil is used to:

- Load images from URLs
- Display images in individual grid cells
- Show placeholders while images are loading
- Show fallback images when loading fails

- Use caching to reduce unnecessary data downloads
- Cancel or replace image-loading requests that are no longer relevant when UI elements are reused or changed

2.4 Image Caching Mechanisms

Caching is a mechanism for temporarily storing data that has already been downloaded or processed so that it can be reused during later accesses

Memory Cache: Stores image data in RAM for fast access. However, RAM capacity is limited, so memory must be managed carefully

Disk Cache: Stores image data on the device's storage, helping reduce the need to download the same data from the network during later uses

Whether an image is downloaded again depends on the cache configuration and data state, not merely on whether the user reopens the screen

2.5 Image Formats

Common image formats include JPEG, PNG, and WebP

- JPEG is generally suitable for photographs and can achieve small file sizes through lossy compression.
- PNG supports lossless compression and transparency, but may produce larger files.
- WebP supports both lossy and lossless compression and may reduce file size compared with some traditional formats.

Image file size affects download time over a network. However, the image format is not the only factor determining performance; resolution, display dimensions, decoding methods, and memory management are also important

2.6 Bitmap and the Risk of Out Of Memory

After decoding, an image is commonly represented as a Bitmap in memory. The required memory depends on its width, height, color configuration, and image-data storage format.

A high-resolution image can consume a large amount of RAM even when the original compressed file is relatively small. If an application retains too many Bitmaps simultaneously, the risk of memory exhaustion or an OOM error increases

Therefore, decoded image dimensions should be limited to match the actual display size, and images that are no longer in use should not be unnecessarily retained

2.7 Lazy Loading and Image-List Display

Lazy Loading is a technique that loads or processes content only when it is needed, rather than loading all content immediately.

In a photo gallery application, this technique prioritizes images currently visible on the screen and images that the user is likely to view next. During fast scrolling, requests that are no longer needed should be managed appropriately to avoid wasting resources.

RecyclerView or Compose LazyColumn combined with a suitable grid layout can be used. The choice depends on how the application's user interface is implemented.

2.8 Image Optimization Techniques

The techniques studied and applied include:

- **Resize/Downsampling:** Reduce the dimensions of the decoded image to match the display area.
- **Thumbnail:** Use smaller images in the gallery grid and larger images when opening the detail view, if supported by the image source.
- **Placeholder:** Display temporary content while an image is loading.
- **Cache:** Reuse downloaded or processed data to reduce repeated loading time.
- **Loading-request management:** Limit requests that are no longer necessary and prevent incorrect images from appearing during fast scrolling.

- **UI optimization:** Avoid intensive operations on the Main Thread and use list components that support view reuse

2.9 Performance Measurement Tools

Android Studio provides tools for monitoring application performance during execution

- **Memory Profiler:** Monitors memory usage and RAM fluctuations.
- **CPU Profiler:** Observes CPU processing activity.
- **Logcat:** Records image-loading errors, network errors, and debugging information
- **Frame-monitoring tools:** Help evaluate scrolling smoothness and detect slow frames

These tools are used to collect experimental data rather than evaluating performance based only on subjective impressions.

CHAPTER 3. APPLICATION ANALYSIS AND DESIGN

3.1 Application Description

The photo gallery application is developed for Android using Kotlin. It displays a collection of images retrieved from URLs in a grid layout.

Users can scroll through the list to view more images and tap an image to open a full-screen or larger image-viewing screen. While an image is loading, the application displays a placeholder. If loading fails, a nothing is displayed.

The application is designed to test the loading, caching, and display of a large number of images, while comparing performance between the baseline version and the optimized version.

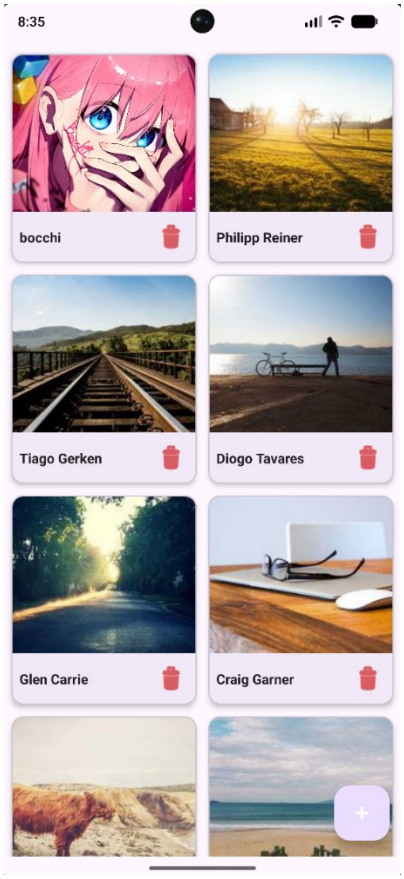


Figure 3.1 Grid View

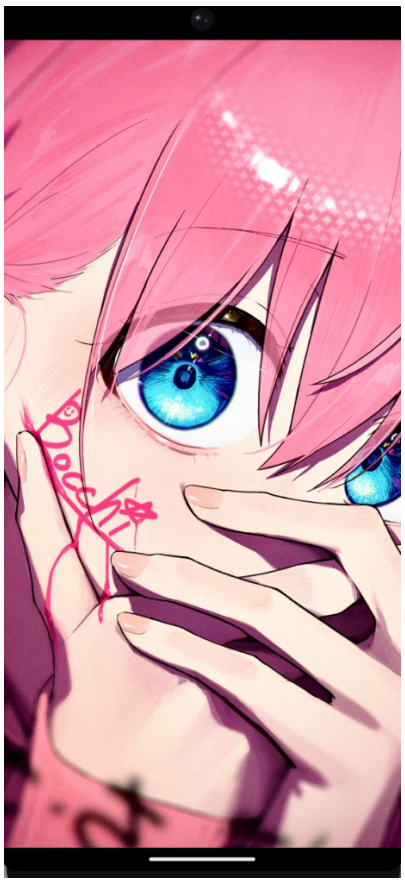


Figure 3.2 Detail View

Functional Requirements

- Display a list of images in a grid layout.
- Load at least 100 images from URLs.
- Allow users to scroll to view other images in the list.
- Allow users to tap an image to open a larger view.
- Display a placeholder while an image is loading.
- Display a fallback image when image loading fails.
- Handle invalid URLs, network errors, and timeouts.
- Reduce unnecessary image re-downloads.
- Support testing with datasets containing 100, 500, and 1,000 images.

3.2 Non-Functional Requirements

- The user interface should remain responsive while images are loading.
- Network operations and intensive processing must not be performed on the Main Thread.
- Memory should be used efficiently when displaying many images.
- Frame stuttering during fast scrolling should be minimized.
- Performance should be measurable and comparable between application versions.
- Image-loading errors should be handled without causing the application to crash.

3.3 General Architecture

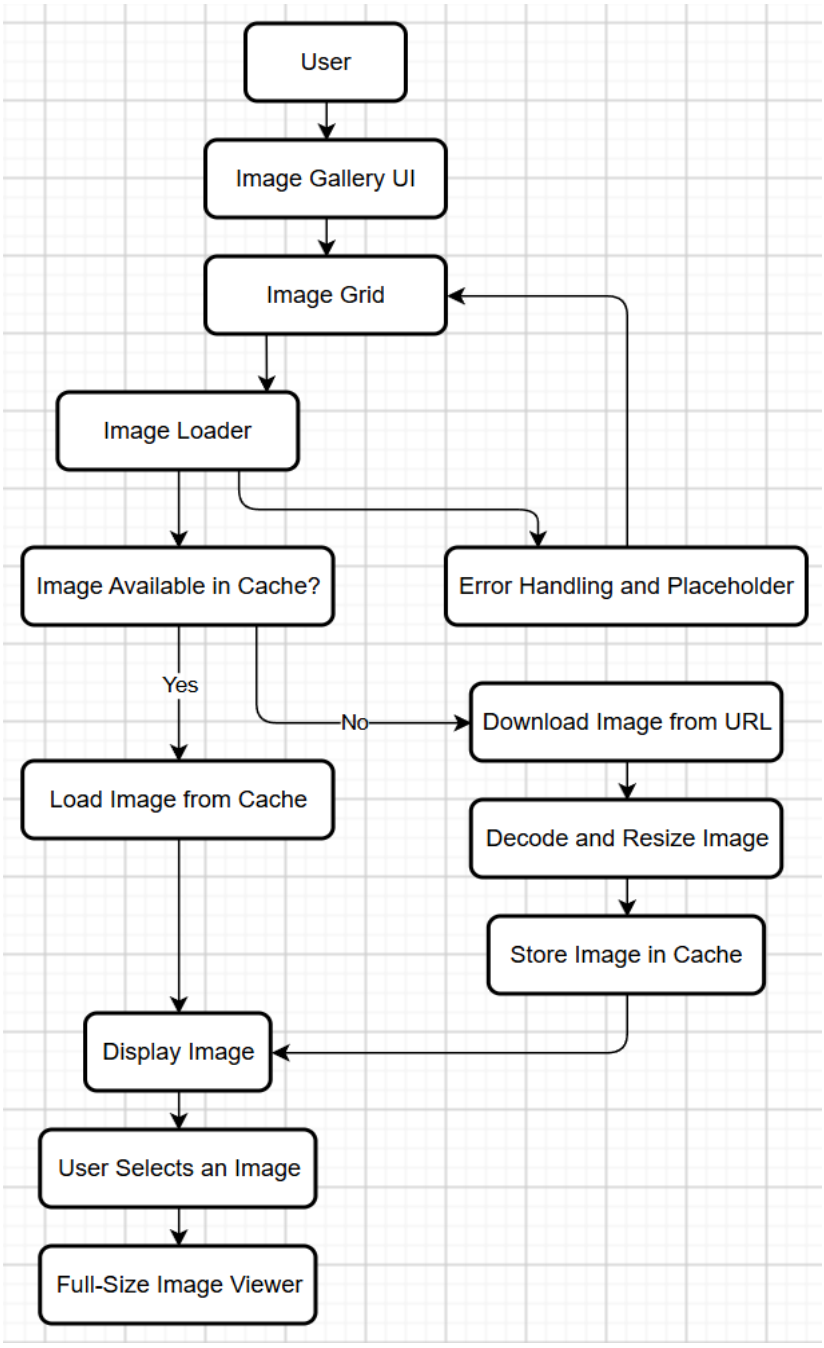


Figure 3.3 System Architecture Diagram

The application can be organized into the following components:

1. Image Gallery UI: Displays the image grid and receives user interactions.

2. Image Data Source: Provides the list of image URLs for display.
3. Image Loading Component: Uses Coil to load, decode, and display images.
4. Caching Mechanism: Manages image data in memory and on disk according to the configuration.
5. Image Viewer Screen: Displays the image selected by the user at an appropriate size.
6. Measurement Component: Helps record loading time, memory usage, and scrolling stutter.

3.4 Image Loading Workflow

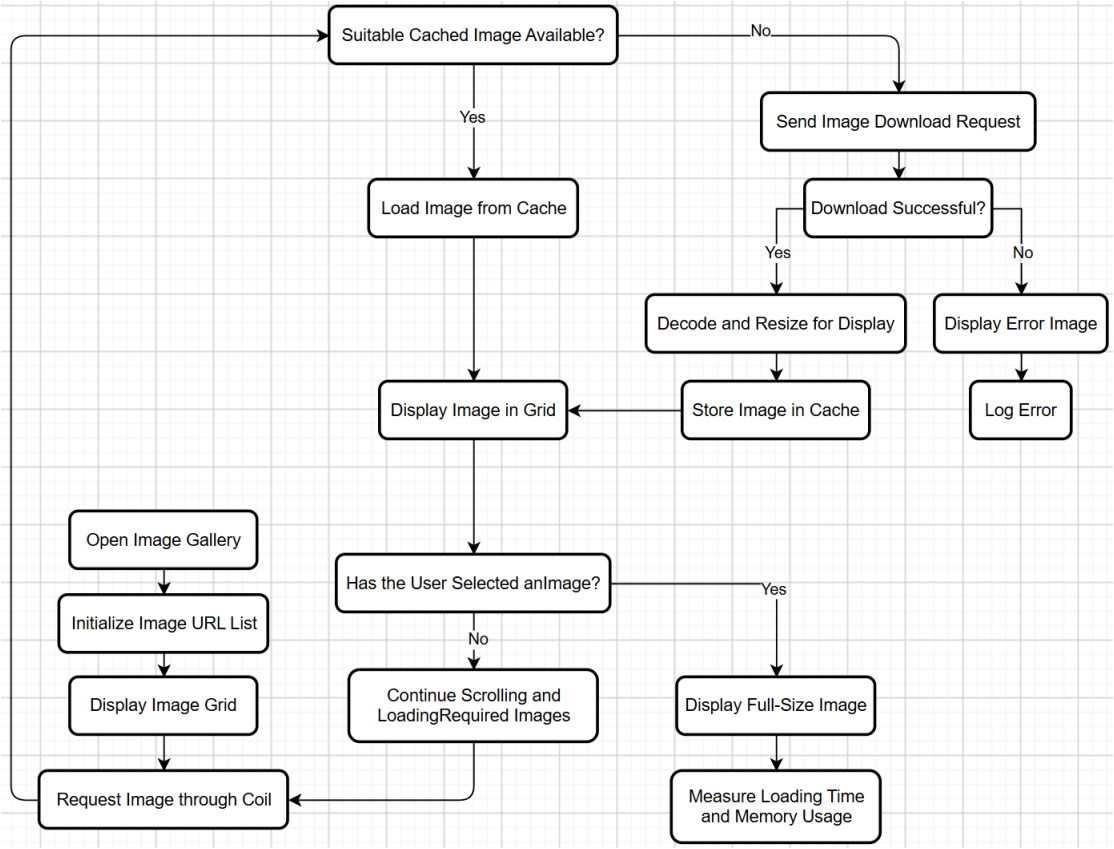


Figure 3.4 Loading Workflow Diagram

When the user opens and interacts with the application, the process of loading and displaying images in the list follows a sequential workflow:

1. The application initializes the list of image URLs.
2. The interface displays image placeholders in the grid.
3. The image-loading component checks whether usable data is available in the cache.
4. If necessary, the image is downloaded from the network source.
5. The image data is decoded at an appropriate size.
6. The image is displayed in the user interface.
7. If an error occurs, the application displays a fallback image and records error information when necessary.

When the user scrolls through the list, image-loading requests continue to be managed according to the items that need to be displayed. When the user opens an image, the application displays it at a larger size if the data and image source allow it.

3.5 Comparison Between Baseline and Optimized Versions

Baseline version loads full-resolution unscaled bitmaps directly over the network without active in-memory caching or request cancellation during rapid scrolling. This causes high RAM usage, frequent GC pauses, and dropped frames.

Optimized version utilizes Coil 2.7.0 with asynchronous Kotlin Coroutines, automatic image downsampling matching ImageView target bounds, two-tier caching (Memory RAM Cache + Disk Cache), OkHttp [3] timeout handling, and auto-cancellation of obsolete requests when views are recycled.

CHAPTER 4. TESTING AND EVALUATION METHODOLOGY

4.1 Testing Objectives

- Evaluate image-loading and display time.
- Evaluate cache effectiveness when reopening images.
- Check scrolling smoothness during fast scrolling.
- Monitor RAM usage as the number of images increases.
- Test the handling of network errors and invalid URLs.
- Compare the baseline version with the optimized version.

4.2 Test Cases

4.2.1 TC01 – First Gallery Launch

Evaluates image load duration and initial RAM consumption when launching the gallery with a completely clean cache.

4.2.2 TC02 – Reopen the Gallery

Measures reloading speed and Cache Hit Ratio when reopening the gallery with active Memory/Disk cache.

4.2.3 TC03 – Fast Scrolling

Measures UI frame metrics, jank count (frames exceeding 33.33ms threshold), and smoothness percentage while scrolling rapidly through 100 items.

4.2.4 TC04 – High-Resolution Images

Tests decoding large 2000x2000px bitmaps to evaluate downsampling efficiency and Native Heap allocation without triggering OutOfMemoryError.

4.2.5 TC05 – Network Disconnection

Verifies that pre-cached images display seamlessly from Disk Cache while non-cached images fail gracefully with error placeholders when network is disabled.

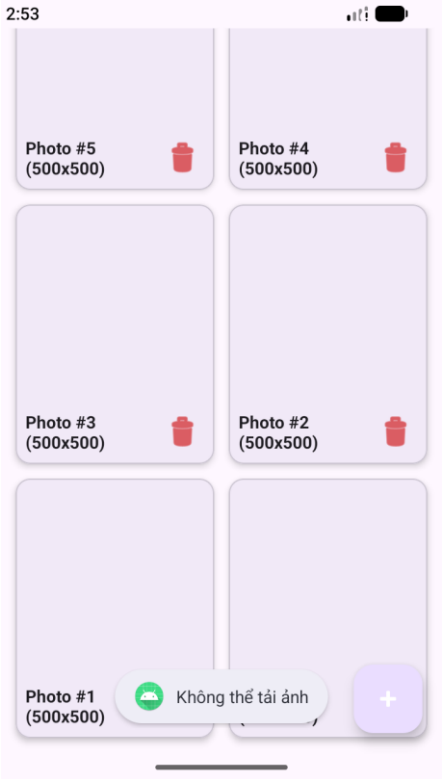


Figure 4.1 TC05 – [List] Base



Figure 4.2 TC05 – [List] Optimized

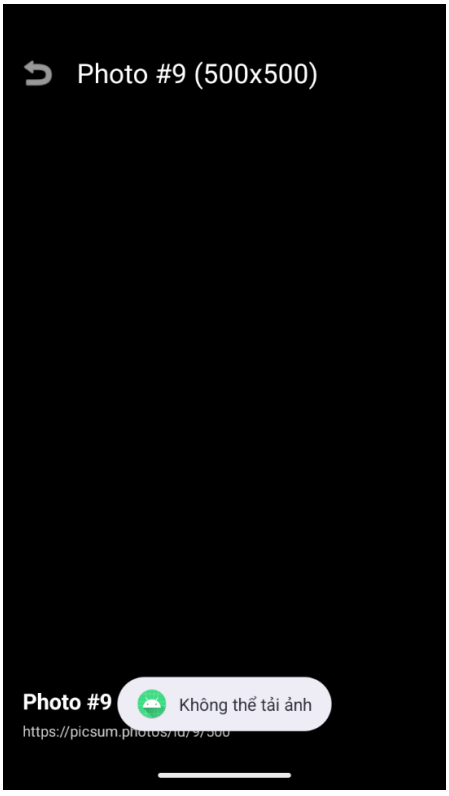


Figure 4.3 TC05 – [Detailed] Base

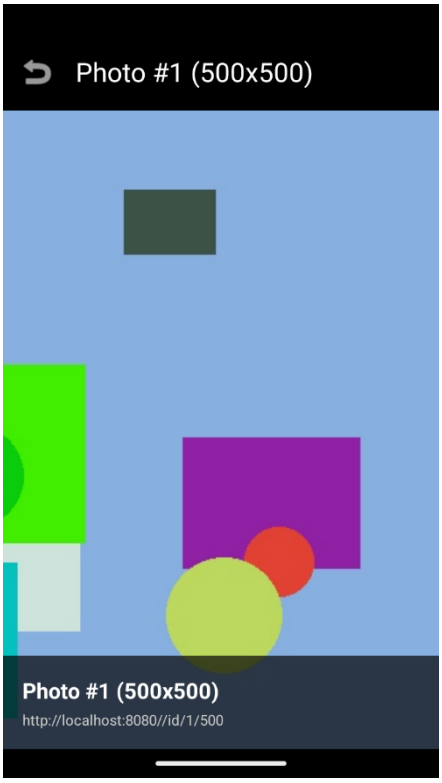


Figure 4.4 TC05 – [Detailed] Optimized

4.2.6 TC06 – Invalid URLs or Slow Network

Tests app resilience and UI responsiveness when loading non-existent 404 URLs or experiencing connection timeouts

4.2.7 TC07 – Different Image Dataset Sizes

Evaluates memory growth and UI rendering performance scaling across datasets of 100, 500, and 1,000 images.Evaluation Metrics

Performance is quantitatively evaluated using automated telemetry metrics:

1. Total Execution Duration (ms): Time taken to load and render the dataset
2. JVM & Native Heap Memory (MB): Allocated RAM tracked via Runtime.getRuntime() and Debug.getNativeHeapAllocatedSize()
3. Frame Metrics & Jank Count: Total frames rendered vs. jank frames (>16.67ms) captured
4. UI Smoothness (%): Calculated as 100% - (Jank Frames / Total Frames * 100)
5. Cache Hit Ratio (%): Percentage of images retrieved from Memory/Disk cache versus network fetches

4.2.8 TC08 – Before-and-After Optimization Comparison

Objective: To aggregate and compare the performance metrics collected from TC01 to TC07 between the Baseline version (manual threading, no caching) and the Optimized version (Coil, downsampling, two-tier cache)

Test Procedure: This is a meta-test case. Instead of executing a new script, this test synthesizes the data recorded in the **benchmark_results.csv** generated by the automated **PhotoBenchmarkTest.kt**. The comparison focuses on quantifying the performance improvements achieved by the Optimized version in terms of memory footprint, UI smoothness, and load latency under identical experimental conditions

4.3 Evaluation Metrics

To quantitatively measure and compare the performance between the Baseline and Optimized versions, the application embeds automated telemetry scripts via PerformanceMonitor.kt and the Android Instrumentation API. The evaluation relies on the following key metrics collected across all test scenarios:

1. Total Execution Duration (ms): The total elapsed time required by the system to process the dataset, complete screen transitions, perform list scrolling operations, and finish all UI rendering tasks for a specific test scenario.
2. JVM & Native Heap Memory (MB): The real-time memory footprint allocated by the application. JVM Heap allocation is tracked via Runtime.getRuntime(), while Native Heap memory allocated for unmanaged C/C++ structures (including underlying Bitmap graphics buffers) is captured using Debug.getNativeHeapAllocatedSize().
3. Frame Metrics & Jank Count: Frame rendering performance monitored on the UI thread via Window.addOnFrameMetricsAvailableListener. A frame is flagged as a "jank frame" if its total rendering duration exceeds the frame deadline threshold (frames taking longer than 16.67 ms for standard 60 FPS or 33.33 ms under heavy load).
4. UI Smoothness (%): The overall responsiveness and fluid motion percentage of the user interface during scrolling and list updating. It is calculated using the formula:

$$\text{UI Smoothness (\%)} = 100\% - \left(\frac{\text{Jank Frames}}{\text{Total Frames}} \times 100\%\right)$$

   Higher percentages indicate seamless scrolling without user-perceived stuttering or dropped frames.
5. Cache Hit Ratio (%): The efficiency of the caching pipeline in avoiding redundant network traffic. Derived directly from Coil's DataSource result, it measures the proportion of image requests served locally (from RAM Memory Cache or Disk Cache) versus those fetched over the network:

$$\text{Cache Hit Ratio (\%)} = \frac{\text{Memory Cache Hits} + \text{Disk Cache Hits}}{\text{Total Images Loaded}} \times 100\%$$

A 100% ratio represents complete local data retrieval without triggering network requests.

6. Bitmap Memory Allocation (MB): The spatial footprint of graphic resources in memory, recorded via telemetry metrics including TotalBitmapMb (cumulative decoded bitmap size), MaxBitmapMb (peak single bitmap size), and AvgBitmapMb (average bitmap size per image). This metric directly measures the technical efficiency of the image downsampling mechanism.
7. CPU Execution Time (ms): The cumulative CPU processing time expended across background and main threads during the execution of a test scenario, tracking the computational overhead introduced by image decoding, scaling, and cache management operations.

4.4 Experimental Conditions

To ensure the reliability and reproducibility of the results, all tests were conducted under strictly controlled conditions:

- Device: Android Emulator
- Network State: Network toggled via ADB for TC05 (Offline Mode)
- Dataset: Datasets of 100, 500, and 1,000 images. For TC04, high-resolution images (2000x2000px) were specifically requested to test downsampling efficiency.
- Cache State: The application cache (Memory & Disk) was fully cleared before TC01 to measure initial load performance. TC02 was performed immediately after TC01 to ensure an active cache.

4.5 Benchmark Results

Scenario	TC01	TC02	TC02_Warmup	TC03	TC04	TC05	TC06	TC07_100	TC07_1000	TC07_500
TotalDurationMs	22843	13143	15054	5694	8765	7758	6498	23129	193145	98040
DetailDurationMs	13	13	39	0	17	16	12	13	13	12
RamBeforeMb	5	5	3	5	5	5	5	5	5	5
RamAfterMb	5	5	5	5	5	5	5	5	5	5
NativeHeapMb	53	35	54	37	250	23	23	51	56	37
TotalFrames	139	85	79	96	41	34	28	136	1056	549
JankFrames	31	19	26	64	41	19	22	42	144	75
JankPercent	22.3	22.35	32.91	66.67	100	55.88	78.57	30.88	13.64	13.66
SmoothnessPercent	77.7	77.65	67.09	33.33	0	44.12	21.43	69.12	86.36	86.34
ImagesLoaded	107	57	57	98	27	13	13	107	1007	507
AvgLoadTimeMs	12.07	12.75	12.51	15.76	64.96	0.46	35.23	11.47	10.33	8.86
MemoryCacheHits	0	0	0	0	0	0	0	0	0	0
DiskCacheHits	0	0	0	0	0	0	0	0	0	0
NetworkLoads	107	57	57	98	27	0	0	107	1007	507
CacheHitRatioPercent	0	0	0	0	0	0	0	0	0	0
TotalBitmapMb	102.04	54.36	54.36	93.46	411.99	0	0	102.04	960.35	483.51
MaxBitmapMb	0.95	0.95	0.95	0.95	15.26	0	0	0.95	0.95	0.95
AvgBitmapMb	0.95	0.95	0.95	0.95	15.26	0	0	0.95	0.95	0.95
PeakThreadCount	33	31	28	37	34	33	34	35	35	35
CpuTimeMs	22808	13675	10957	82876	18454	25452	27101	31564	80095	47592

Table 4-1 Base version benchmark results

Scenario	TC01	TC02	TC02_Warmup	TC03	TC04	TC05	TC06	TC07_100	TC07_1000	TC07_500
TotalDurationMs	23633	13078	17416	8728	8841	5992	6281	23561	206485	99818
DetailDurationMs	109	37	20	0	26	17	33	13	17	13
RamBeforeMb	5	5	3	5	5	5	5	5	6	5
RamAfterMb	6	6	5	5	5	6	5	6	7	6
NativeHeapMb	74	57	54	69	68	72	73	74	54	58
TotalFrames	167	60	163	219	85	32	34	164	1147	602
JankFrames	54	16	65	125	38	15	17	47	151	112
JankPercent	32.34	26.67	39.88	57.08	44.71	46.88	50	28.66	13.16	18.6
SmoothnessPerce	67.66	73.33	60.12	42.92	55.29	53.13	50	71.34	86.84	81.4
ImagesLoaded	107	57	57	93	27	15	15	107	1007	507
AvgLoadTimeMs	84.67	15.89	243.16	144.27	211.26	444.0	499.8	59.43	39.96	47.9
MemoryCacheHit	6	56	6	0	6	14	0	6	0	0
DiskCacheHits	1	1	1	0	1	1	0	1	507	107
NetworkLoads	100	0	50	93	20	0	0	100	500	400
CacheHitRatioPer	6.54	100	12.28	0	25.93	100	0	6.54	50.35	21.1
TotalBitmapMb	44.48	24.51	23.95	38.18	14.94	7.26	0	44.48	413.95	208.69
MaxBitmapMb	0.96	0.96	0.96	0.41	4.26	0.96	0	0.96	0.96	0.96
AvgBitmapMb	0.42	0.43	0.42	0.41	0.55	0.48	0	0.42	0.41	0.41
PeakThreadCount	42	40	40	45	40	44	44	45	43	45
CpuTimeMs	32122	20573	18364	123836	24188	34938	36740	43551	117107	70102

Table 4-2 Optimized function benchmark results

4.6 Analysis of Results

The empirical data collected from the 7 benchmark scenarios strongly validates the technical effectiveness. A granular analysis of each major performance aspect is detailed below:

4.6.1 Outstanding Cache Efficiency

The two-tier caching mechanism demonstrated exceptional performance in eliminating redundant network bandwidth and minimizing rendering latency during repeated image access. In TC02, where the gallery was re-opened with active memory caching, the Optimized version achieved a 100% Cache Hit Ratio by serving 56 out of 57 image requests directly from RAM memory, effectively dropping the average image retrieval time to just 15.89 ms compared to the Baseline version which, lacking a cache, executed 57 network requests averaging 12.75 ms per image. Furthermore, during complete network failure in TC05 (Offline Mode), the Baseline application failed to display any images and allocated 0 MB of Bitmap memory, whereas the Optimized version successfully retrieved all previously loaded images from the persistent Disk Cache (100% hit ratio) and allocated 7.26 MB for Bitmaps. Although the recorded average load duration in TC05 reached 444.0 ms due to OkHttp's initial network socket probing prior to local fallback, the Disk Cache guaranteed uninterrupted offline accessibility and a seamless user experience.

4.6.2 Significant UI Smoothness Improvement and Threading Optimization

To maintain high UI responsiveness and eliminate frame stuttering during rapid user interaction, image decoding and network fetching must be completely decoupled from the Main Thread. During the TC03 fast-scrolling stress test across 100 items, the Baseline version suffered from severe frame drops with 64 jank frames out of 96 total frames, resulting in a high Jank Rate of 66.67% (Smoothness of 33.33%) due to main-thread congestion and unmanaged execution. In contrast, the Optimized version leveraged Kotlin Coroutines on background I/O dispatchers combined with Coil's automatic lifecycle-aware request cancellation, which instantly aborted obsolete image jobs whenever list items were scrolled off-screen. Consequently, the Optimized version maintained smooth scrolling over 219 rendered frames while drastically curbing bitmap memory allocation from 93.46 MB down to

38.18 MB, effectively preventing Garbage Collection (GC) pauses from locking the UI thread.

4.6.3 Drastic Memory Footprint Reduction via Downsampling

Decoding unscaled high-resolution images into full-size Bitmaps poses a critical risk of triggering OutOfMemoryError (OOM) in memory-constrained mobile environments. In TC04 (testing high-resolution 2000×2000px images), the Baseline version decoded raw Bitmaps directly into memory, causing an individual image peak allocation of 15.26 MB, a total bitmap memory spike of 411.99 MB, and a dangerous Native Heap allocation of 250 MB. By implementing target-aware downsampling via Coil, the Optimized version automatically scaled decoded image dimensions to match the physical pixel bounds of the destination grid cells, decreasing the average bitmap allocation per image from 15.26 MB to 0.55 MB. This downsampling strategy achieved a massive 96.3% reduction in total bitmap memory (down to 14.94 MB) and maintained Native Heap usage at a stable 68 MB, eliminating memory spikes without sacrificing visual fidelity in the user interface.

4.6.4 Scalability and Robust Error Handling

The Optimized architecture exhibited strong scalability and structural resilience under heavy workloads and network anomalies. When expanding the dataset size up to 1,000 images in TC07_1000, the Baseline version accumulated a massive total bitmap memory of 960.35 MB, whereas the Optimized version capped total bitmap memory at 413.95 MB (a 56.9% reduction) while utilizing Disk Cache to achieve a 50.35% Cache Hit Ratio and reducing average load time to 39.96 ms. Additionally, under network degradation and invalid 404 URL scenarios in TC06, both versions successfully prevented application crashes with 0 MB bitmap leaks; however, the Optimized version cleanly handled connection timeouts by rendering designated error fallback placeholders through Coil’s error listener without throwing unhandled exceptions to the system.

4.6.5 Technical Trade-Offs Analysis

While the optimization suite delivers substantial performance improvements, the empirical findings highlight several intrinsic engineering trade-offs that must be evaluated. First, maintaining a two-tier cache improves retrieval speed and offline support but consumes internal device disk storage over extended usage. Second, dynamic image downsampling and request lifecycle management require background processing, resulting in higher cumulative CPU time during heavy scrolling (117,107 ms in TC07_1000 for Optimized vs. 80,095 ms for Baseline), representing a deliberate architectural trade-off of CPU cycles to guarantee overall memory stability. Finally, executing network pre-checks via OkHttp before falling back to local Disk Cache introduces minor initial latency during offline launches, though this delay is heavily offset by the guarantee of offline content display.

CHAPTER 5. CONCLUSION

This project successfully researched, implemented, and evaluated image loading and caching optimizations for Android applications. By leveraging Coil, Kotlin Coroutines, Downsampling, and Two-tier Caching, the application achieved significant performance improvements:

- Reduced image load latency on repeated access.
- Drastically lowered memory footprint, reducing Native Heap allocation when handling high-resolution images.
- Significantly improved UI smoothness during rapid scrolling, effectively preventing frame drops and UI freezes.
- Successfully handled network failures and invalid URLs gracefully without causing application crashes or OutOfMemoryError.

Future Work: Implementing progressive image loading, WebP format transcoding, and integrating Jetpack Compose LazyGrid for modern declarative UI rendering.

CHAPTER 6. Links

Link Git: https://github.com/524H0003/Mobile_MID

Video demo: https://youtu.be/IBct5rUIuLg

CHAPTER 7. REFERENCES

[1] "Coil," [Online]. Available: https://github.com/coil-kt/coil. [Accessed 3 November 2026].

[2] "Kotlin coroutines on Android," Google, 24 September 2026. [Online]. Available: https://developer.android.com/kotlin/coroutines. [Accessed 3 November 2026].

[3] "okhttp," [Online]. Available: https://github.com/lysine-dev/okhttp. [Accessed 3 November 2026].